Dynamic Graph

CS3014 - Estructura de Datos Avanzados

Luciano A. Romero Calla
lromeroc@utec.edu.pe

2026-1

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Overview

1. Dynamic Connectivity
2. Euler-Tour Trees

1.

Dynamic Connectivity

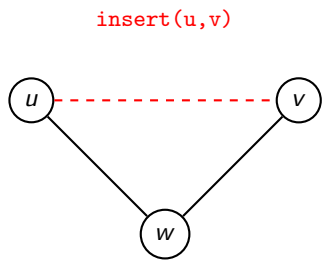
Dynamic Connectivity

Goal

Maintain an undirected graph subject to updates and queries.

Operations

- `insert(u, v)`: Add edge (u, v).
- `delete(u, v)`: Remove edge (u, v).
- `connected(u, v)`: Are u and v in the same connected component?



Problem Categorization

Fully Dynamic

Admits arbitrary interleavings of edge insertions and deletions.

Incremental / Decremental

Partially dynamic variants restriction to *only* insertions or *only* deletions.

Tree-Structured Solutions: A Contrast

Before considering general graphs, we resolve dynamic connectivity within **forests**.

Metric / Feature	Link-Cut Trees (Sleator-Tarjan)	Euler-Tour Trees (Henzinger-King)
Primary Layout	Heavy-Light Decomposition	Flattened Traversal (Euler Tour)
Aggregate Bounds	Path Aggregates ($O(\log n)$)	Subtree Aggregates ($O(\log n)$)
Implementation	Complex splay-tree interlinking	Single balanced BST representation
Edge Queries	Difficult to locate arbitrary edges	Trivial lookup via sequence positions

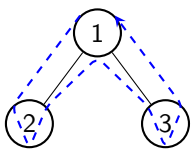
2.

Euler-Tour Trees

Euler-Tour Tree Mechanics

Concept: Convert a tree T into a sequential format by tracking its traversal boundary. Every undirected edge is spanned exactly twice (downward and upward).

- Each node visit maps to an index in a sequence.
- Sequence backed by a balanced BST (e.g., Treap, AVL).
- Keys are implicitly defined by sequential array order.



Sequence: $1 \to 2 \to 2 \to 1 \to 3 \to 3 \to 1$

Structural Mutations: Link & Cut

By tracking pointers to the first and last occurrence of node v, we manipulate subtrees in $O(\log n)$ time via basic BST splits and concatenations.

Cut(v)

Removes the subtree rooted at v.

1. Identify v's first and last visit in the BST.
2. Split sequence into three: *Before-v*, *Subtree-v*, *After-v*.
3. Concatenate *Before-v* and *After-v*.

Link(u, v)

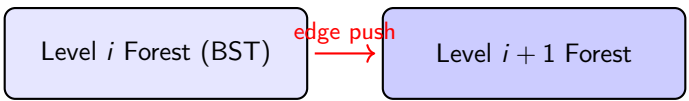
Attaches u's tree as a child of v.

1. Split v's sequence right before its final occurrence.
2. Splice the entire sequence of u into the gap.
3. Merge everything back into a unified BST.

Fully Dynamic Connectivity in General Graphs

How do we handle general graphs rather than isolated trees?

- Maintain spanning forests of the graph.
- Partition edges into $\log n$ levels based on assigned "weights" or frequencies.
- Use an Euler-Tour tree to maintain the spanning forest of each level.
- **Amortized Cost:** $O(\log^2 n)$ for updates, $O(\log n)$ for queries.



Summary & Lower Bounds

Key Takeaways:

- Euler-tour trees are simpler to implement and analyze than Link-Cut trees for aggregate subtree information.
- General graph dynamic connectivity can be achieved in $O(\log^2 n)$ time using level-partitioned Euler-Tour trees.

Lower Bounds (Pătraşcu & Demaine):

- $O(x \log n)$ update requires $\Omega(\log n / \log x)$ query time.
- $O(x \log n)$ query requires $\Omega(\log n \log x)$ update time.
- *Conclusion:* There is an inherent trade-off in dynamic graph structures!

Thanks

References

Acknowledgements

The course slides are based on the lectures from previous editions and on similar lectures elsewhere. List of credits: Erick Demaine, Keith Schwarz